基于 ZYNQ 的双核双 DDR 大语言模型端侧推理系统

摘要

本系统面向端侧大语言模型推理，在 ZYNQ UltraScale+ MPSoC ZU7CG 开发板上构建双核双 DDR 软硬件协同方案。PS 运行 PYNQ 4.0，完成分词、模型装载、寄存器控制与网页交互；PL 内 core0、core1 按张量分片并行执行 Llama3.2-3B-Instruct 的主要计算。core0 连接 PS DDR，core1 连接 PL DDR，通过核间 AllGather 与 AllReduce 交换向量和部分和。

系统采用双核张量分片、数据流调度和双存储通路，结合 Llama3.2 的模型几何、位置编码、十七位 Token 编号与 GQA 结构配置运算和接口。采用四位分组权重、八位 KV 缓存，通过离线展开 KV 头配合物理头调度，重排模型镜像以支持 8192 Token 硬件上下文。每核使用四组三十二路 FP16 通道，分阶段复用完成二十八层计算。

250 MHz、8192 上下文版本已完成布局布线、静态时序检查和 bit 生成，建立了与 HWH、模型镜像配套的 PYNQ 工程、终端诊断与网页界面。已接受候选的建立、保持和脉宽裕量分别为 +0.020 ns、+0.001 ns、+0.017 ns。该配置的板端吞吐率、长上下文和实测功耗仍需验证；已有 187.5 MHz、1024 上下文版本的文本推理记录可作迁移基线。

第一部分 作品概述

1.1 功能与特性

系统在板端存放模型并执行自回归文本生成，支持单轮 Prompt、保留历史的 Chat、最大输出设置和 UTF-8 输入输出。网页顶部展示硬件状态、时钟、首 Token 延迟和速度，左侧对话、右侧日志；权重上传使用百分比、进度条和 MiB/s 提示。用户从 Jupyter 终端启动服务后点击打印的网址进入界面，可加载加速器、上传双 DDR 模型，并在空闲状态切换到独立空闲 bit，重新演示启动过程。模型语言能力与生成质量取决于所用权重，不以界面支持多语言代替模型质量评测。

1.2 应用领域

本系统适用于受限网络环境下的本地文本交互、轻量问答、文档摘要，以及 FPGA 大模型部署教学和软硬件协同研究。模型、输入与主要计算驻留端侧，可用于观察存储访问、核间通信、算子复用及上下文长度对系统的影响。面向具体业务时，仍需补充相应数据集、生成质量、响应时延和供电散热测试；报告不把潜在应用写成已完成的行业部署。

1.3 主要技术特点

采用双核张量分片与 PS/PL 双 DDR 架构，CPU 控制路径和模型上传路径分开。core0 的四路 128 位 HP 与 core1 的一路 512 位 AXI 提供两组访存通道，核间链路按阶段完成归约或汇聚。W4 权重在引擎前解包，KV8 缓存降低历史键值存储开销；同一组 FP16 主阵列在投影、历史注意力和前馈阶段复用，归一化及部分累加保留 FP32 路径。8192 镜像与硬件清单共同约束布局，软件同时校验上下文预算、内存预留和硬件文件配套。

1.4 主要性能指标

表 1 主版本规格与实现状态

项目	250MHz 8192 版本
器件	xczu7cg-ffvc1156-1-e
模型	Llama3.2-3B-Instruct
核心与存储	2 核 双 DDR
核心时钟	250 MHz
硬件上下文	8192 Token
权重与 KV	W4 / KV8
每核镜像	2,063,458,304 字节
实现状态	bit 已生成 时序通过

1.5 主要创新点

本系统围绕模型、硬件和部署进行协同设计：通过 KV 头离线展开、RoPE 配置与十七位 Token 接口支持 Llama3.2-3B；保持量化权重字节不变，重排模型镜像与 KV 空间以支持 8192 上下文；将硬件生命周期、权重上传进度、Chat

和诊断整合为可迁移的 PYNQ 工程。

1.6 设计流程

先核对模型几何、量化与存储布局，再通过 Scala 参数化描述生成 RTL，配套构建 PS、双 DDR 与控制互联。完成局部接口验证后开展综合、布局布线和时序审阅，导出匹配的 bit、HWH 与清单。软件按相同配置生成双核镜像，部署 PYNQ 运行库与网页服务，再通过寄存器、DDR、文本生成和重复请求分级验证。精简交付包从已生成 RTL 开始复现。

第二部分 系统组成及功能说明

2.1 整体介绍

系统由离线准备环境和板端运行环境组成。离线环境完成模型导出与 Vivado 工程实现；板端由 PS、PL 加速核、PS DDR、PL DDR 和网页服务组成。推理期间，CPU 处理文本及控制命令，计算核从各自 DDR 读出参数和缓存，核间链路合并张量分片。两个核心各自保有 LM Head 和 Greedy 选择路径，完整词表的部分得分归约后，仅 core0 将结果提交 CPU。

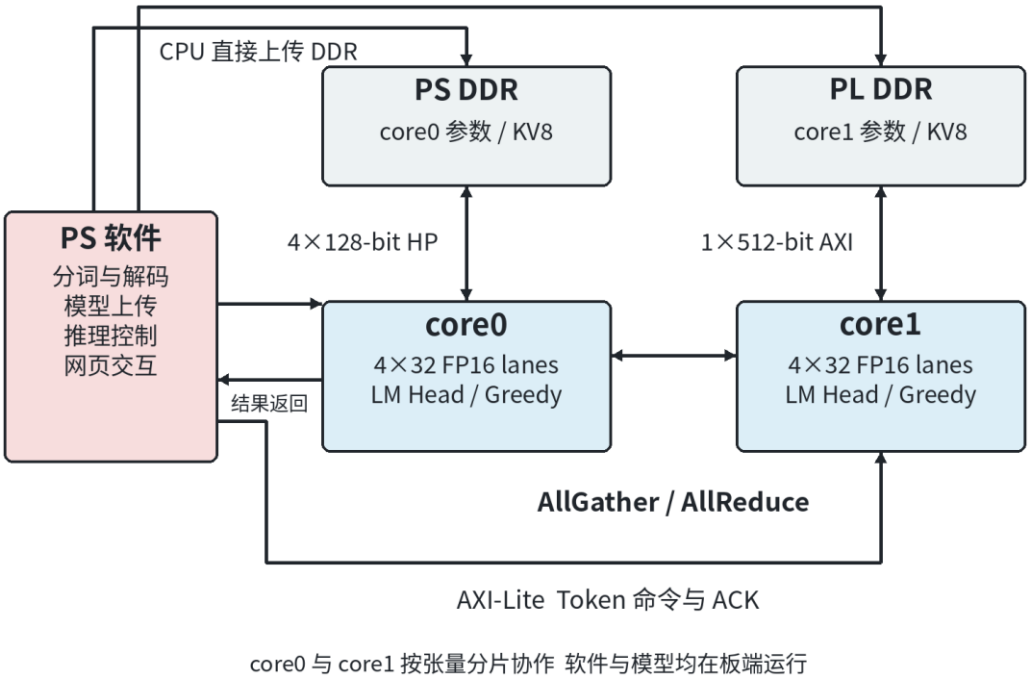


图 1 PS 双核与双 DDR 的系统关系 箭头表示逻辑数据路径

模型参数见表 2。Llama3.2 模型卡提供模型系列与使用背景[1]，具体几何和

部署上限以本项目清单为准。8192 为当前硬件的上下文预算，输入提示词、Chat 历史和预留输出共同占用该预算；不能把模型原始标称上下文直接视为本加速器容量。

表 2 模型与硬件几何参数

参数	配置值	工程含义
隐藏维度 / 层数	3072 / 28	两核协同执行全部层
查询头 / 原生 KV 头	24 / 8	离线重复三次为 24 个物理头
每核物理头 / 头维度	12 / 128	按头调度与分片
MLP 中间维度	8192	Gate Up Down 几何
词表规模	128256	输入输出需 17 位 Token
量化分组	128 元素	四位权重与零点 FP16 缩放
最大位置预算	8192 Token	配套缓存和镜像布局

2.2 硬件系统介绍

2.2.1 硬件整体介绍

系统面向同型号 ZU7CG 开发板，在 PYNQ 4.0 环境中部署 Llama3.2-3B-Instruct。PS 负责分词、运行控制和网页交互，PL 内的 core0、core1 按张量分片协作完成二十八层计算。core0 通过四路 128 位 HP 接口访问 PS DDR，core1 通过一路 512 位 AXI 接口访问 PL DDR，核间链路承担向量汇聚和部分和归约。 器件资源和 PS/PL 基本能力参见 AMD 数据手册[2]。

本版本核心时钟为 4 ns 周期，即 250 MHz。PS 控制、加速核和 DDR 接口涉及不同时钟及复位域，部署时使用同一实现导出的硬件描述，不能仅在软件中改写显示时钟。整体资源统计包含加速核、存储控制与互联，不能直接解释为单核资源。

core0 模型镜像加载到 PS 物理地址 0x800000000 起的专用 2 GiB 区域，启动设备树以 no-map 将其从操作系统常规内存中排除。适配层把核心内部 0x36000 逻辑基址映射到该物理起点。core1 镜像经 CPU 的 0x400000000 起始窗口写入 PL DDR；该访问窗口为 4 GiB，镜像布局使用的逻辑池为 2 GiB，二者需区分。

8192 版本每核镜像为 2,063,458,304 字节，包含 Embedding、各层参数、KV

缓存和量化元数据预留区。扩容通过重排缓存及后续区域偏移实现，原有量化权重字节保持，不重新量化。两份镜像分别存放于 PS DDR 和 PL DDR，187.5 MHz 与 250 MHz 的 8192 硬件共享同一镜像布局，不兼容 1024 镜像。

表 3 板端地址与内存用途

CPU 物理区域	起始地址	范围与用途
PS 模型预留区	0x800000000	2 GiB no-map
PL DDR 访问窗口	0x400000000	4 GiB CPU 窗口 模型逻辑池 2 GiB
加速器控制窗口	0x500000000	AXI-Lite 命令 状态 结果

2.2.2 机械设计介绍

系统采用现有 ZU7CG 开发板及其原有安装、供电与散热结构，未将定制外壳、机械加工或母板设计列为本项目成果。展示时应固定板卡和存储连接，保持散热通畅，并在实物照片中标注电源、网络、PS DDR 和 PL DDR 的实际位置。

2.2.3 电路模块与核内数据流

PL 系统由双核 DataPath、PS DDR 访问适配、PL DDR 控制、AXI 互联、AXI-Lite 控制和时钟复位网络组成。AXI 数据通道承载权重、向量和 KV，AXI-Lite 承载命令与 Token 编号，核间链路根据标签路由归约和汇聚数据。CPU 的模型上传通过地址窗口进入 DDR，不经过乘加阵列。

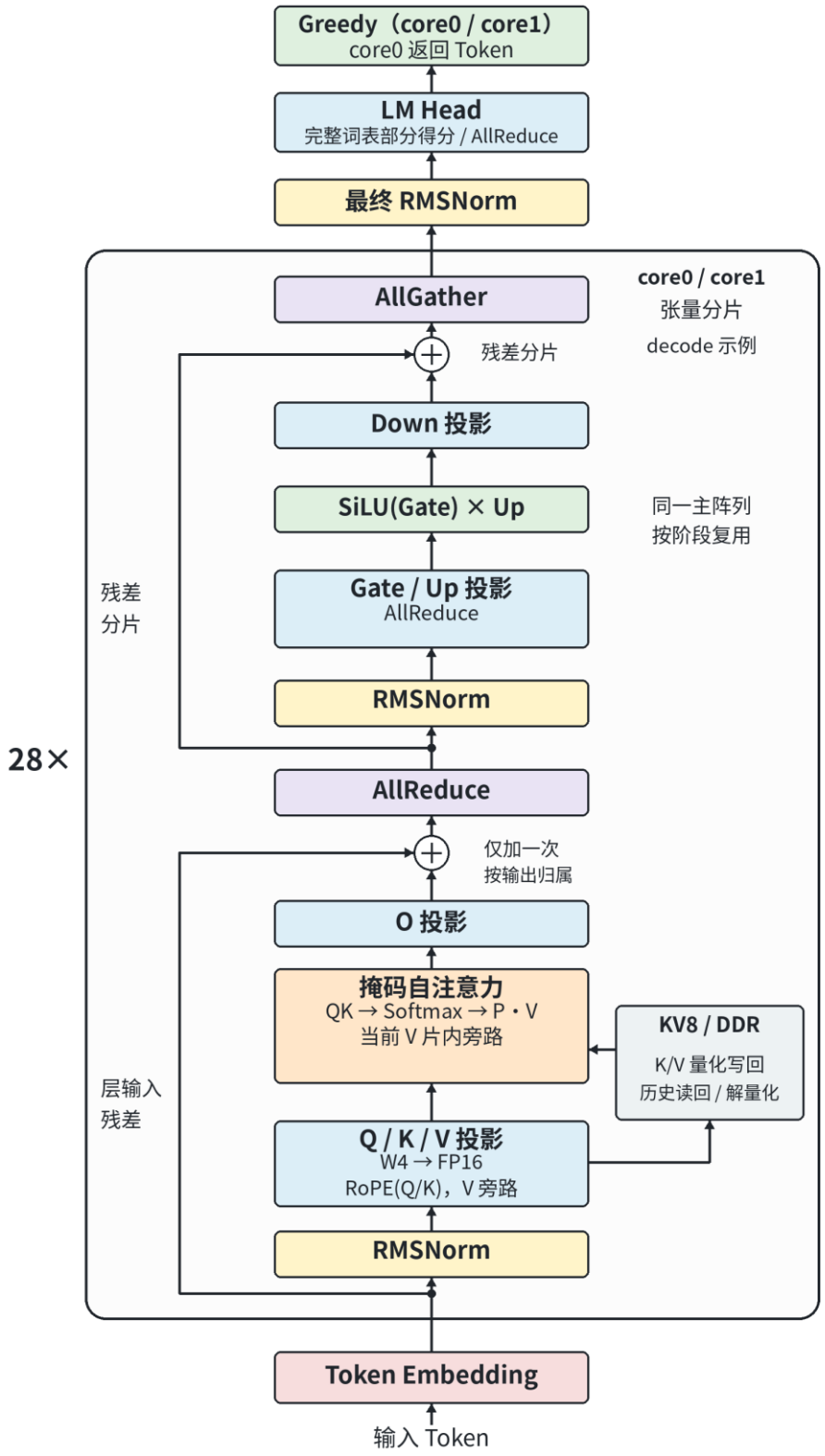


图 2 Llama3.2 解码器层结构与双核协作

以已有历史的解码 Token 为例，输入向量先经过 RMSNorm，再由 W4 拆包、减零点和分组缩放路径向主引擎提供数据，按头计算 Q、K、V。每核主阵列包含 128 条 FP16 通道，划为四组，每组三十二条；同一阵列在多个阶段复

用。Q、K 按 Llama3.2 的位置编码参数进行前后半向量配对旋转，V 经旁路进入后续阶段。 RMSNorm 与 RoPE 的算法背景分别参见文献[4]和[5]。

新的 K、V 经八位量化后连同缩放参数写入对应 DDR，历史缓存读回并解量化。当前 Q 按头维度缩放后与历史 K 点积，形成各位置分数；现有 FP16 Softmax 路径减去最大值后将数值下限裁为 -16，再求指数与归一化。概率与各位置的 V（含当前位置）加权累加，得到注意力输出。每核处理十二个物理头，缓存按展开后的物理头保存，不采用原生 GQA 缓存共享。

注意力输出进入 O 投影，在指定核心仅加入一次残差，再经核间 AllReduce 合并部分和。随后再次归一化，计算 Gate、Up 并归约，经 SiLU 激活与逐元素乘法形成门控结果。Down 投影采用现有 AXPY 路径，分片加入残差后通过 AllGather 汇聚，供下一层使用。完成二十八层后，两个核心计算完整词表的部分 logits，归约后贪心选择下一 Token。

PS 与加速器通过 AXI-Lite 交换命令和状态，控制空间位于 CPU 地址 0x500000000。Llama3.2 的 128256 词表需要十七位 Token 编号，因此设置独立高位与完整结果寄存器。提交时先写 0x28 的最高位，再写 0x00 的低十六位及 prefill、decode 命令；结果有效后从 0x2C 读取完整编号，并写 0x80 确认接收。

推理分为 prefill 与逐 Token decode，CPU 将输入编号广播给两个核心，读取 core0 返回的采样结果并继续下一轮。模型上传与计算控制分开：软件先完成硬件配置，再依次写入两块 DDR，后续生成持续复用驻留模型。日常运行保留文件大小、硬件配套和内存预留检查；详细诊断入口额外提供寄存器快照、权重哈希和 DDR 回读，便于定位故障。

表 4 主要数据与控制接口

接口或寄存器	输入或输出	作用
PS HP / PL AXI	模型向量 KV 读写	core0 四路 128 位 core1 一路 512 位
核间互联	向量或部分和	AllGather / AllReduce 按阶段选择
0x28 与 0x00	Token 高位 低位及命令	支持完整 17 位词表编号
0x04 与 0x2C	状态与完整结果	轮询有效位后读取 Token
0x80	ACK	确认接收 推进生成

2.3 软件系统介绍

2.3.1 软件整体介绍

PYNQ 工程分为配置、硬件生命周期、物理内存访问、寄存器控制、分词与生成、终端交互和网页服务等模块。统一入口支持独立 prompt、保留历史的 chat 和网页 UI；用户可设置最大输出，输入及回答采用 UTF-8，语言能力取决于本地模型。chat 将历史重新编排为输入，提交前校验历史、当前提示词和预留输出是否共同超出所选硬件上下文。 PYNQ 的 Overlay 与 MMIO 提供硬件下载和寄存器访问能力[3]。

PC 浏览器仅承担交互展示，板卡网页服务调用同一套 Python 运行库。服务支持配置选择、初始化、生成、日志状态同步和清除 bit；多个浏览器连接共享服务状态。Chat 每轮软复位并重新提交完整历史，保留对话文本不等同于在硬件中跨轮复用 KV。

2.3.2 模型导出与运行模块

模型导出使用项目实际实现的 affine_u4_include_zero_rne 方案，按一百二十八个元素分组保存四位权重、四位零点和 FP16 缩放参数，KV 缓存采用八位格式。原模型具有二十四个查询头和八个 KV 头，为与物理头调度匹配，在离线导出时将每个 K、V 头复制为三份，共二十四个物理头。该方法保持现有调度接口，但会增加权重及缓存占用。

导出器按核心划分张量与镜像区域，并保存模型几何、量化规则、偏移和内容哈希等元数据，供部署配置核对。8192 扩容版本从原镜像搬移不可变参数区域，重新安排 KV 与元数据空间，并将新增缓存初始化为零。运行时根据所选上下文匹配相应镜像和 bit、HWH；网页界面中的版本切换同时关联硬件及模型布局，避免只替换 bit 而沿用不匹配的镜像。

网页界面顶部显示加速器状态、上传进度和性能指标，左侧对话、右侧日志；终端启动后打印板卡实际访问网址。权重上传显示百分比、已传字节和速度，日常模式不做权重哈希或 DDR 回读。清除 bit 操作在空闲时加载独立空闲配置，供重新展示初始化流程。部署文档覆盖同型号新板的设备树、虚拟环境、模型与硬件配置。

表 5 软件模块输入与输出

模块	关键输入	关键输出或作用
模型打包	模型配置 张量 上下文	core0.bin core1.bin 与清单
配置与 Overlay	硬件组合 bit HWH	地址 时钟 版本及硬件初始化
物理内存装载	镜像 路径 目标 DDR	上传进度和驻留模型
Tokenizer 与 Chat	UTF-8 提示词 对话历史	Token 序列及上下文预算
寄存器生成	Token 序列 最大输出	逐 Token 回调 终止原因
日志与计时	阶段状态 时间戳 字节数	TTFT token/s MiB/s 日志
网页生命周期	用户设置 初始化或清除	共享状态和空闲配置切换

2.4 精简工程的复现流程

2.4.1 复现对象与输入条件

复现以 llama32_3b_ctx8192_250mhz 精简工程为入口[6]，目标为 xczu7cg-ffvc1156-1-e、250 MHz、8192 Token 上下文和双核双 DDR。可直接部署随包硬件，也可从包内 RTL 重新完成 Vivado 实现；两种路径分别验收。更换 FPGA 型号、板级 DDR 接线或 PS 外设配置时，不能仅更换软件配置后沿用本 bit。

工程分为 src、build 和 board/runtime 三部分。src 保存已生成的 RTL、IP、约束及工程脚本；build/build.sh 与 variant.json 定义构建入口和版本；board/runtime 保存 bit、HWH、硬件清单、Tokenizer、运行库、网页和 test 诊断入口。包内没有 Scala 源码、模型二进制、历史实现检查点或离线依赖安装包。因此，此包能重建已有 RTL 的实现，不能据此重新生成 Scala RTL；模型和依赖需另行准备。

2.4.2 在编译主机重建 Vivado 工程

主机需安装 Vivado 2025.2 及 Zynq UltraScale+ MPSoC 所需器件和 IP 支持，并具备 Bash 与 Python 3.9 或更新版本。进入精简工程根目录，再运行构建。以下路径为工具安装位置示例，应替换为实际路径；--jobs 1 限制并行运行任务数，不等于所有内部线程均为单线程。

```
source /path/to/Vivado/2025.2/settings64.sh
sha256sum -c SHA256SUMS
bash build/build.sh --stage bitstream --jobs 1
```

bitstream 阶段会依次创建工程、生成 IP 与 Block Design 输出、综合、实现并写出 bit 和 XSA。需要分阶段检查时，可依次运行以下命令。默认 generate 指 IP/BD 输出生成；impl 停在布局布线后物理优化，尚未生成 bit，必须继续运行 bitstream。

```
    bash build/build.sh
bash build/build.sh --stage synth --jobs 1
bash build/build.sh --stage impl --jobs 1
bash build/build.sh --stage bitstream --jobs 1
```

产物位于 build/work/llama32_3b_ctx8192_250mhz，包括 XPR、运行日志、实现目录、reports 和同名 XSA。脚本采用 Explore、AggressiveExplore 及布线后物理优化策略。LLAMA_BUILD_SUCCESS 只表示脚本完成，不能作为时序通过判据；应独立审阅 timing_impl、check_timing_impl、clocks_impl、cdc_impl 和 drc_impl 等报告，核对 WNS/TNS、WHS/THS、WPWS、未约束路径及相关例外。重新布局的结果不能继承既有候选的正裕量。

随包 hw 中的 bit、HWH 与 manifest.json 是已配套发布的硬件组合，可用于直接部署。重新构建后，应从同一次工程输出重新取得 bit 与 HWH，重新生成与新 bit、HWH 匹配的完整 manifest，包含 bit 头、文件记录、地址、时钟及来源证据，再进行部署验收。精简包尚未提供上述三件套的自动发布工具，不能只替换 bit 后沿用旧 HWH 或旧清单并宣称完成新版本发布。

2.4.3 在同型号新板配置 PYNQ 运行环境

将 board/runtime 完整复制到已启动 PYNQ 4.0 的同型号 ZU7CG 板卡。按工程根目录的权重下载说明取得 8192 版本 core0.bin、core1.bin，放入运行目录的 data 下；每份必须为 2,063,458,304 字节。1024 上下文镜像与此布局不兼容。文件包含权重和缓存预留，不能仅按模型名称判断是否匹配。

在写入模型前，启动设备树必须把 PS 高位 DDR 的 [0x800000000, 0x880000000) 声明为独占的 2 GiB no-map 区域。需先确认该范围确实对应板卡物理 DDR，再修改实际引导使用的设备树并重启，检查运行中的 /sys/firmware/devicetree/base。该预留不是 CMA 或交换分区，也不能在 Python 运行时临时划出。已有其他保留节点时须核对重叠；精简包不包含自动改写启动设备树的脚本。

在根节点与 reserved-memory 的 #address-cells、#size-cells 均为 2 且一致，reserved-memory 的 ranges 为空的前提下，模型子节点可表示如下。它仅说明本工程的地址约定，不是适用于所有 ZU7CG 镜像的完整设备树补丁。

```
    llama32-ps@800000000 {
    reg = <0x8 0x0 0x0 0x80000000>;
    no-map;
};
```

板端使用与 Jupyter 一致的 PYNQ 虚拟环境。本例为 ARM64、Python 3.12，默认解释器位于 /usr/local/share/pynq-venv/bin/python3；首次运行还需 cc 或 gcc 编译内存复制辅助程序，并具有项目所需的物理内存访问权限。Tokenizer 需要 transformers 4.45 或更新版本，不要求安装 PyTorch。联网时可安装 requirements.txt；离线时需另备匹配 ARM64 与该 Python 版本的依赖，精简包本身不含 wheel。

```
    # 以下命令均在板端复制后的 runtime 目录执行
export PYNQ_PYTHON=/usr/local/share/pynq-venv/bin/python3
# 仅在板卡联网时安装依赖
"$PYNQ_PYTHON" -m pip install -r requirements.txt
"$PYNQ_PYTHON" tools/check_board_environment.py
"$PYNQ_PYTHON" -m llama32_pynq check
```

环境检查命令退出成功仍需阅读输出，确认 ps_reservation.ready、硬件组合、模型长度与 Tokenizer 等项目均满足要求。启动设备树错误、镜像不匹配或文件缺失应先处理，再下载加速器和上传模型。

2.4.4 分级上板验证与日常运行

日常演示从网页入口启动，终端打印当前板卡可访问的网址；在电脑浏览器打开即可，无需配置 DP 桌面。页面先初始化硬件，再显示双 DDR 权重上传的进度、百分比和 MiB/s，随后输入提示词并设置最大输出。Chat 将历史一起计入 8192 上下文预算，界面选项不能扩大硬件缓存。权重上传完成后驻留 DDR，继续对话无需每轮重新上传。

```
    bash run.sh --mode ui --host 0.0.0.0 --port 8088
# 二选一：终端交互也可使用同一运行库
bash run.sh --mode chat --max-new-tokens 128
```

日常 UI 与 Chat 仅上传权重，不做权重哈希和 DDR 全量回读。首次部署或排障使用 test 中的日志入口，依次验证普通文本、同进程软复位后的相同输

入，再验证两个不同提示词；记录逐 Token 编号、首 Token 延迟、生成用时和解码速度。诊断可能包含权重校验和回读，耗时不应混入模型生成性能。

```
    bash test/run_terminal_validation.sh --skip-bos \
  --prompt 'Explain what an FPGA is.' --max-new-tokens 16
bash test/run_same_session_repeat.sh \
  --prompt 'Explain what an FPGA is.' --max-new-tokens 16
bash test/run_two_prompts.sh \
  --prompt-1 '解释一下 FPGA。' --prompt-2 'What is a CPU?' \
  --max-new-tokens 64
```

第一条诊断命令显式跳过 BOS-only，后两条直接测试文本输入，不执行 BOS-only，因为其重复性问题仍需单独定位。文本合理、相同输入重复一致、不同输入可运行和接近上限的长输入，是不同的验证项目；其中一项通过不能代替其余项目。现有旧版文本日志可用于回归对照，本节提供复现步骤，250 MHz、8192 版本的实板结果需单独采集，不能从工程精简与文件齐备推导吞吐率。

2.5 自定义板卡 PYNQ 构建与经验固化

2.5.1 构建对象与方法来源

PYNQ 镜像是运行加速器的基础环境。为把自定义板卡适配中的排障过程变成可复用方法，将板级检查、构建恢复、烧录启动和无线驱动经验整理为 pynq-build skill[7]。每条规则尽量对应实际失败或观察、最小修正、验证记录和适用限制，使后续移植能够说明“为何这样做”，而不是重复一组缺少条件的命令。

案例主要来自 MyZU7 与 RKZU15EG 的 PYNQ 4.0 构建，工具为 Vivado/Vitis 2025.2，EDF manifest 为 amd-edf-rel-v25.11.1。下述问题大多发生于 RKZU15EG foundation-r2，作为镜像构建方法的证据，不计作本报告 ZU7CG 加速器的上板成果。其他 PYNQ 版本、Zynq-7000 或不同 EDF 分支，应先核对其真实构建入口，不能直接套用这些依赖和补丁。

2.5.2 从硬件输入到可恢复构建

新板适配首先记录仓库版本、板卡型号、XSA 哈希、工具链、EDF manifest、板级配置及构建日志。XSA 可生成基础硬件描述，但板外 PHY、复位、电源、时钟和接口模式仍需原理图或实测补充。DDR、SD/eMMC、UART、以太网和 USB 的配置必须对应具体板卡，不以同系列 SoC 作为相同引脚、电压或外设连接的依据。

本例采用 EDF/SDT 路径，由 boards 下的板卡 .spec、edf.env 及配套设备树定义输入；不能未经检查改用旧 PetaLinux BSP 流程。.spec 声明 aarch64、FPGA Manager 与所需阶段软件包；edf.env 关联 manifest、custom 模式、目标 machine 和 XSA。名称及路径应按已建立的板卡目录填写，machine 必须与配套生成配置对应，不能只改字符串。

在已配置工具链挂载的编译容器中，主机仓库映射为 /workspace，可按以下形式调用 sdbuild。BOARD_NAME 需替换为实际板卡目录名。该命令说明构建入口，不包含 Docker、授权或首次依赖准备的全部过程。

```
    cd /workspace/sdbuild
BOARD_NAME=MyBoard
env -u REBUILD_PYNQ_SDIST -u REBUILD_PYNQ_ROOTFS \
  make BOARDS="$BOARD_NAME"
```

本地 Makefile 使用 ifdef 判断变量是否存在，而非数值真假；将 REBUILD_PYNQ_ROOTFS 设为 0 仍会选中重建分支。因此不需要重建时应移除变量。失败恢复也应保留当前工作目录及匹配的 downloads、sstate 和工具链缓存，先找第一条实际错误；末尾 make Error 或 mconf missing 可能只是后续结果，不宜直接 make clean。清理会破坏增量状态，且可能移除输出镜像。

恢复规则的边界由输入和脚本语义决定：硬件、设备树或内核配置变更后，需要重新生成受影响阶段；旧 RK wrapper 支持 --resume，而 foundation-r2 入口不接收参数，部分 hook 仍清理指定 boot 配方，因此不能统一宣称从精确失败点续编。BitBake 下载缓存、sstate、工具链及 apt/pip 缓存也各有范围，fetch 离线通过不等于后续全部阶段可离线完成。

2.5.3 从失败与观察形成的具体规则

（1）普通设备树编译通过仍可能在增强流程失败。RKZU15EG 的一处 endpoint 写法能通过普通 dtc，却在 EDF/Lopper 增强后出现“Properties must precede subnodes”。将对应片段展开为规范的多行结构后，普通编译仍通过，增强编译由失败转为通过，基础 DTB 哈希保持不变。enhancement-results.json 保留了前后对照。该组局部回归没有执行域转换，不能单靠此记录证明完整域生成；这次格式修正不证明 USB、DP 已在板端工作，也不是所有设备树 warning 的通用消除方法。

（2）模块链接失败需回到目标器件配置。内核 MODPOST 曾报告 mmi-dc.ko 缺少 DMAengine PCM 相关符号。该驱动面向案例板不需要的 Versal Gen2 显示路径，因此仅关闭 CONFIG_DRM_AMD_MMI_DC，并保留 ZynqMP DPSUB 配置。olddefconfig-results.json 显示目标差异集中于这一选项，后续内核和模块构建通过。结论是去除本板不适用的驱动选项，而非关闭整个显示系统；它不能证明 DP 显示或音频已实测正常。

（3）产物选择应按用途识别而非按文件名猜测。ZOCL 打包阶段同时匹配到 updates/zocl.ko 与 .debug/zocl.ko，造成“应只有一个模块”的检查失败。修正为从实际配方输出中选择运行模块并排除调试副本，不按时间戳任取一个。真实 package-split 产物重放和十四项选择、拒绝测试通过，记录于 real-recipe-replay.json 与 report.json。十四项属于局部测试，不是十四次完整构建，也不代表 ZOCL 已在目标板加载。

（4）内存验收器也可能把正确布局判为错误。DDR 检查曾要求低位 A53 域完整暴露 2 GiB，但实际少 1 MiB。核对真实 DTB、SDT 地址映射及保留属性后，确认该差额来自已知 PMU 保留范围。修正只允许这一明确例外，不增大设备树中的物理内存；十二项正反检查同时覆盖合法布局、其他缺口、重叠及高位区域，高位 2 GiB 条件仍保持。ddr-validator-evidence.json 支持的是校验规则正确性，不是 DDR 压力测试；该例外也不替代本加速器额外需要的高位 no-map 模型预留。

（5）相同内核名称不足以证明驱动可复用。MyZU7、旧 RK USB-host 和 foundation-r2 的 uname -r 均为 6.12.40-xilinx-g0900176227cc，但 .config 与 Module.symvers 不同。AIC8800 因此按目标镜像的 kernel-devsrc、配置和符号表重新编译；缺少 asm/types.h 时先准备对应的 ARM64 内核生成头文件，不能链接主机 x86 头文件，modules_prepare 本身也不能生成完整符号表。

foundation-r2 的两份无线模块完成 MODPOST、架构、vermagic 与导出符号检查，分别解析六十五和二百二十个导入符号，证据保存在 BUILD-INFO.json 和 build-validation.json。该内核未启用 MODVERSIONS，因此不能将符号解析描述为 CRC 兼容证明；记录中的 runtime_load_test 仍为 NOT_RUN。旧 USB-host 镜像曾出现 wlan0、完成无线关联并取得 DHCP 地址，只能作为旧组合的

实测，不能移作 r2 的运行结论。

（6）SD 读回差异需定位到数据含义。一次 cmp 的首处差异位于第 109052973 字节。转成零基偏移并减去根分区起点 109051904 和 ext4 超级块偏移 1024，得到 44，对应 s_mtime 挂载时间字段。这说明自动挂载可能改变镜像元数据，不能仅凭 cmp 非零判定写卡损坏；但也不能只解释首处差异便放行全部差异。需要继续区分访问时间、校验和、内部 journal 与未知内容，不能将 journal 所在区域等同于事务已验证。

该次记录中的只读 e2fsck -fn 完成五轮检查并返回 0。在未知差异已排除、文件系统结构检查通过的前提下，可尝试上板启动；这不证明文件负载或整卡逐字节一致。skill 的只读比较脚本以 0 表示镜像长度内一致，3 表示限定挂载字段变化，4 表示受支持布局中的时间、校验和及 journal 区域变化，1 表示其他差异或失败；3、4 都不是完全一致。工具目前限定 MBR 第二分区为 ext4 等已实现布局，其他格式应说明不支持。复杂差异归类仅在已经出现 cmp 差异时启用，不增加为每次烧录的固定负担。

（7）存储与启动故障应按链路分层定位。同一张卡在内置读卡器上出现 error -22、tuning failed 和第 0 扇区 I/O error，无法读出分区表；改用 USB 读卡器后又能识别两个分区。这个对照支持先排查读卡链路，不足以宣判卡永久损坏，也不证明随后写卡和启动成功。设备名随读卡器变化，实际写入前仍需现场核对目标。串口端口可打开只说明 USB 串口桥可访问，不能证明 PS 已启动；应按端口、115200/8N1、供电、启动模式和早期引导继续定位。

2.5.4 有效性验证与适用边界

skill 的有效性按证据强度逐层确定：先用同一输入的失败与修正后结果验证局部规则，再确认真实构建消费了相应输入，随后核对镜像、启动文件与驱动的对应关系，最后由板端登录和功能测试确认实际运行。局部正例说明目标故障被解除，反例用于防止规则放宽后接受错误输入；二者都不能跳过下游构建与上板验收。表 6 汇总当前证据的边界。

表 6 PYNQ 构建经验的验证层级

层级	已有验证	本层不能单独证明
局部规则	增强编译前后对照；ZOCL 十四项；DDR 十二项	真实构建继续使用修正后的输入
构建与打包	内核和模块构建；r2 最终静态检查通过	目标板启动及各外设实际功能
写卡与结构	差异偏移分析；只读文件系统检查返回 0	未知差异排除；实际引导结果
板端功能	旧 MyZU7 有运行记录；旧 RK 有无线关联与 DHCP 获址记录	r2 成功登录、无线加载及其他板端功能

foundation-r2 最终报告状态为 FINAL_BUILD_STATIC_CHECKS_PASS，同时明确 physical_board_test=NOT_RUN_BY_THIS_SCRIPT。后续还存在上电串口无输出的记录，因此不能写成“自定义板卡全部移植成功”。MyZU7 的历史启动和无线扫描、旧 RK 的无线关联与 DHCP 获址、r2 的静态通过分属不同镜像，均按各自硬件、配置和时间保留，避免将验证结论跨版本拼接。

最终固化的是基于证据选择下一步的方法：保留可恢复状态，针对第一条真实错误修改，用同输入对照与必要反例验证，再把结论限制在测过的层级。没有发生过的边界或并发情形不默认扩展为复杂代码；只有新的日志、硬件变化或复现实验给出依据时，才增加相应检查。这样既能复用经验，也能避免把个案补丁变成所有板卡必须携带的配置。

2.6 时序优化过程与验收

时序优化采用先建立 187.5 MHz 基线、再解决 250 MHz DDR 接口瓶颈、最后完成 8192 上下文增量实现的路线。模型和上下文变更会形成不同 RTL 版本；各版 RTL 确定后，主要调整布局、布线与物理优化策略。250 MHz 版本采用 4 ns 核心约束，候选之间保留目标时钟及原有 MIG 检查，不通过删除路径、放宽阈值或修改逻辑来替代物理收敛。失败候选和旧工程均独立保留，便于比较[8]。

2.6.1 建立 1024 上下文的低频基线

1024 / 187.5 MHz 原 impl_2 的 WNS 为 −0.075 ns，不能因负值较小便作为合格结果。从布线前检查点建立独立候选，采用 AggressiveExplore 布线和

Explore 路由后物理优化，将最终 WNS 改善到 +0.042 ns，同时 WHS 为 +0.010 ns、WPWS 为 +0.017 ns。该候选作为功能迁移和物理策略的基线，原 impl_2 及其失败证据继续保留。

2.6.2 从 DDR 数据关系定位升频瓶颈

升至 250 MHz 后，早期 routed 快照的 WNS 为 +0.015 ns、WHS 为 +0.010 ns，但 DDR MIG 内部仍有 −0.097 ns 的 MinSkew 违例。失败位于 RXTX_BITSLICE 的 D[2] 相对 D[1] 数据检查，关联 MIG 的 mmcm_clkout0，周期约 3.332 ns；它不是核心 4 ns 时钟的普通高低脉宽问题。尽管该项列在 pulse-width 报告中，setup 和 hold 为正也不能消除这项失败。

由此把验收范围扩展到真实 MinSkew 覆盖与数值，而不只看 WNS。处理前先核对两条数据支路的驱动、到达时间和物理延迟，不把 −97 ps 直接当作某一条网络需要增加的延迟，也不据此修改核心时钟占空比。问题分析与最终候选分别存档，避免把中间快照当成最终结果。

2.6.3 从局部修补转向全局布局优化

局部重布线、寄存器位置调整等试验表明，MinSkew 与 setup 存在实际取舍：修复相对到达关系可能使数据路径变长，进而恶化建立时间。于是从 post-opt 检查点重新布局，采用 Explore 布局、AggressiveExplore 物理优化及布线、Explore 路由后优化，再执行带 preserve 的 NoTimingRelaxation 重布线。每个候选仍检查完整时序，不能只看被修复的端点。

1024 / 250 MHz 最终 WNS 为 +0.003 ns，WHS 为 +0.008 ns，全设计 MinSkew 最小余量为 +0.301 ns。这里的 +0.301 ns 是最终设计的全局最小值，不表示最初 −0.097 ns 的同一 pin 对恰好改善到该数值。上述策略是这个候选实际执行的物理实现流程，不是所有重新构建均能复现同一裕量的保证。

2.6.4 通过增量物理参考完成 8192 版本收敛

8192 / 187.5 MHz 先通过独立布局布线取得 +0.114 ns WNS。随后对 8192 / 250 MHz 进行对照：普通 Explore 最终 setup 通过，但 MinSkew 为 −0.694 ns；完整 NoTimingRelaxation 对照仍有 −0.061 ns WNS 和 −0.258 ns MinSkew。采用已验收的 8192 / 187.5 MHz 物理参考后，MinSkew 转正，WNS 仍为

−0.170 ns，说明低频结果的高复用率本身不能保证高频目标收敛。

最终从当前 8192 / 250 MHz post-opt 设计出发，以已验收的 1024 / 250 MHz DCP 作为增量物理参考，使用 TimingClosure 导入匹配的放置与布线信息，保留当前 8192 网表、4 ns 约束、PS 配置和接口。实际流程为增量导入、Explore 布局、AggressiveExplore 物理优化及 Explore 布线；参考 DCP 不替换当前网表，也不将 8K 功能退回 1K。

该增量候选最终取得 WNS +0.020 ns、WHS +0.001 ns、WPWS +0.017 ns，MinSkew 最小余量为 +0.078 ns。通过后没有追加新的布局布线或额外路由后物理优化，而是从已接受的确切 DCP 生成 bit。精简包不包含历史参考 DCP，其重建入口是从 RTL 重新实现；若要复现本节增量路径，还需使用完整归档中的对应检查点和脚本，并重新验收。

2.6.5 检查覆盖与最终硬件绑定

后期观察到重新打开 DCP 后，MinSkew 报告覆盖曾减少，因此仅有“重开后无负值”不足以证明通过。最终采用实际布线会话的完整报告，比较首次现场、最终及保存后同会话结果，确认十一处物理位置的 Slow/Fast 共二十二条记录完整，检查启用及原阈值不变。覆盖完整与裕量合格分别验收，同时检查 setup、hold、脉宽、总线偏斜、全布线状态和 DRC。

CDC 与边界告警按实际端点、同步结构及约束逐项审阅，保留自动报告和具体说明，不把厂商 IP 层次作为整体豁免。接受记录绑定 DCP、报告和来源哈希，再由该 DCP 写出 bit，核对配套 HWH 与清单。四个已归档候选的汇总见表 7；TNS、THS、TPWS 均为 0，8192 / 250 MHz 的保持裕量仅 1 ps，属于余量较紧的合格结果。

表 7 四版已归档候选的静态时序结果

上下文 Token	时钟 MHz	WNS ns	WHS ns	WPWS ns
1024	187.5	+0.042	+0.010	+0.017
1024	250	+0.003	+0.008	+0.017
8192	187.5	+0.114	+0.009	+0.017
8192	250	+0.020	+0.001	+0.017

本节结论对应已归档的具体实现、约束和候选，不延伸为所有新布局都能通

过。重新构建、更换器件或修改接口后应重新核对完整检查；板端功能、长上下文推理、稳定性与实测性能仍单独验证，不能从静态时序表直接推导。

第三部分 完成情况及性能参数

3.1 整体介绍

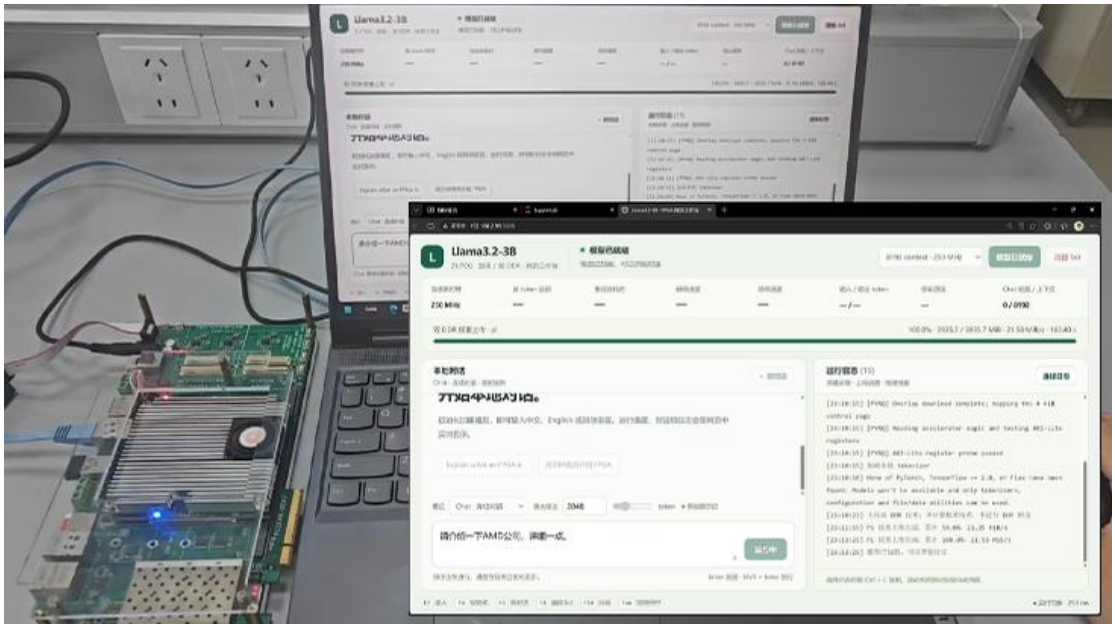


图 3 系统连接与网页界面

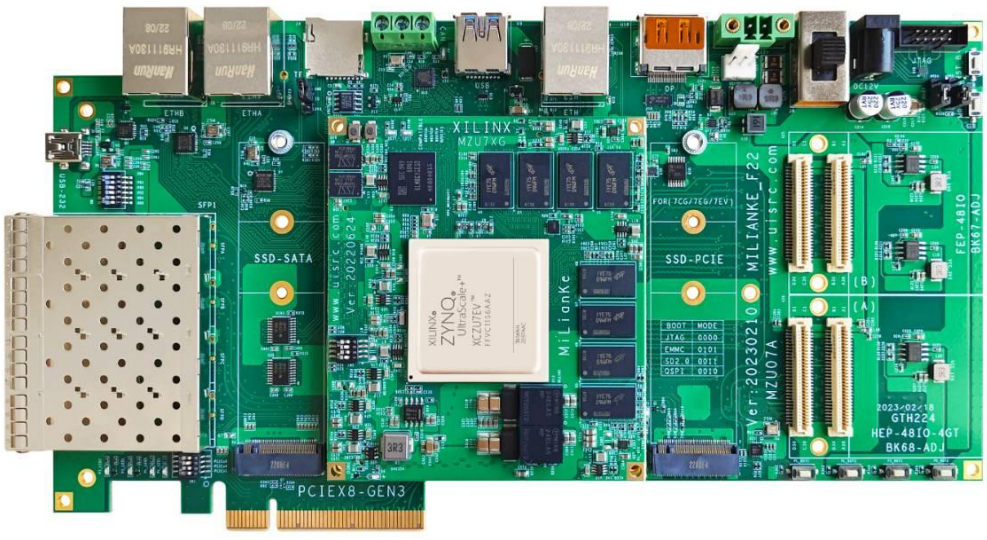


图 4 同系列板卡接口布局示意 图示器件为 XCZU7EV

3.2 软件成果

已交付终端运行、Chat、网页 UI、带日志的 test 诊断目录、离线依赖与新板部署文档。网页包含硬件组合、最大输出、状态、双 DDR 进度、对话和日志；清除操作加载独立空闲 bit，保留磁盘上的模型与加速器文件，不宣称擦除全部 DDR。图 3 展示系统连接与网页界面，主版本的板端性能仍按实际测试补充。

3.3 特性成果与量化指标

表 8 250MHz 8192 版本静态实现与功耗估算结果

指标	主版本实现结果
核心时钟周期	4.000 ns 250 MHz
WNS / TNS	+0.020 ns / 0 ns
WHS / THS	+0.001 ns / 0 ns
WPWS	+0.017 ns
bit 生成与 HWH 配套	已完成
Vivado 估算功耗	约 13 W

表 8 的时序数据来自已接受候选的实现报告。正的时序余量描述该检查点在当前约束下的建立、保持和脉宽结果，不代替板端运行。保持裕量为 0.001 ns，后续改变布局、频率或约束时应重新验收，不沿用本次结果。约 13 W 为 Vivado 功耗估算，不是整板实测；估算使用的活动率、温度与供电条件应随功耗报告保存。

表 9 全设计布局布线资源统计

资源	使用量	可用量	占用率
CLB LUT	178649	230400	77.54%
CLB Registers	270506	460800	58.70%
Block RAM Tile	177	312	56.73%
UltraRAM	12	96	12.50%
DSP	623	1728	36.05%

模型与缓存共同占用镜像空间。8192 版本每核约 1.922 GiB，均置于各自 2 GiB 逻辑池中；它包含缓存预留，不能直接作为纯权重大小。GQA 的离线三次展开保持硬件兼容，也增加物理 KV 存储，尚未采用原生 KV 共享。

第四部分 总结

4.1 可扩展之处

后续首先在同型号新板上核对 250 MHz 时钟、双 DDR 访问与配套镜像，分阶段测试相同提示词重复生成、不同提示词和接近上下文上限的输入。每项记录首 Token 延迟、解码吞吐率、温度与功耗，再据实补充新版性能指标。

在基线验证稳定后，可评估原生 GQA 缓存共享以降低重复存储，或优化较长输入的处理效率。这些方向尚未实现，应先给出资源与协议影响，再修改 Scala 并通过局部仿真和实现时序检查，保持模型计算、存储布局和控制协议的一致性。

4.2 心得体会

工程开发需要同时管理模型、硬件和运行软件的版本。模型名称相同并不代表镜像布局一致，频率选项也不能替代 bit 与 HWH 的版本匹配。独立配置、模型清单和可追溯报告用于记录各版本的参数、接口及实现变化，减少定位问题时需要同时排查的范围。

验证应把功能、静态时序与实际性能分开记录。正的时序余量只能说明当前约束下的实现结果，单次输出合理也不足以证明重复生成和长上下文全部正常。局部仿真适合定位已观察到的寄存器、复位或缓存问题；上板日志则提供输入、逐 Token 结果和时间指标，二者共同构成可复查的技术依据。

上下文扩容也需要从存储布局出发。把上限从 1024 改为 8192，会改变每头缓存跨度、后续权重偏移和模型镜像长度，界面中一个数字不能替代这些变化。通过保持不可变权重字节、单独搬移缓存区域，并将模型与硬件配置关联，可以使扩容过程保持可解释、可复查。

工程实现应围绕已经观察到的问题组织修改与验证。修改 Scala 后先检查生成的接口和局部模块，再审阅实现时序，最后用板端输入输出与日志确认行为。对没有测量的数据保留测试条件和空缺，有助于在后续演示与新板部署中准确判断完成程度。

第五部分 参考文献

[1] META. Llama 3.2 3B Instruct Model Card[EB/OL]. 2024[2026-11-01]. https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct.

[2] AMD. Zynq UltraScale+ MPSoC Data Sheet Overview DS891[EB/OL]. 2025[2026-11-01]. https://docs.amd.com/v/u/en-US/ds891-zynq-ultrascale-plus-overview.

[3] XILINX. PYNQ 4.0.0 Verona Release[CP/OL]. [2026-11-01]. https://github.com/Xilinx/PYNQ/releases/tag/v4.0.0.

[4] ZHANG B, SENNRICH R. Root Mean Square Layer Normalization[C]//Advances in Neural Information Processing Systems. 2019. https://arxiv.org/abs/1910.07467.

[5] SU J, LU Y, PAN S, et al. RoFormer Enhanced Transformer with Rotary Position Embedding[EB/OL]. 2021[2026-11-01]. https://arxiv.org/abs/2104.09864.

[6] 项目技术资料. llama32_3b_ctx8192_250mhz 精简工程及 PYNQ 运行说明[CP]. Version1/output_lite/llama32_3b_ctx8192_250mhz, 2026[2026-11-03].

[7] 项目技术资料. 自定义板卡 PYNQ 编译与适配 skill[CP]. outputs/pynq-build-skill/pynq-build/SKILL.md 及 references、构建验证记录, 2026[2026-11-03].

[8] 项目技术资料. 250 MHz DDR MinSkew 分析与 8192 上下文实现记录[CP]. Version1/validation/llama32_250m_routed_pulse_analysis.md 及 ctx8192_250mhz_20261101/ACTIVE_BUILD.md, 2026[2026-11-03].